Plataforma Inteligente de Control de TMO

Auditoría operativa y automatización de reportes horarios para Call Centers / BPO · README del proyecto · v1.0.0

1. Nombre y descripción

Nombre del proyecto

Plataforma Inteligente de Control de TMO, Auditoría Operativa y Automatización de Reportes Horarios para Call Centers (paquete: `call-center-tmo-backend` / `call-center-tmo-frontend`).

Qué es y para qué sirve

Aplicación web para empresas BPO que carga archivos Excel con registros de llamadas, calcula el **TMO** (Tiempo Medio de Operación, AHT) por asesor y global con ponderación por volumen, clasifica a cada asesor con una regla determinista, responde consultas en lenguaje natural sobre cualquier asesor y envía por correo, cada 60 minutos, el balance de asesores que exceden el umbral frente a los que cumplen, con un libro Excel adjunto.

Problema que soluciona

- **Trabajo manual:** supervisores dedican de 3 a 5 horas diarias a descargar, limpiar y recalcular tablas de telefonía (PBX/ACD).
- **"Promedio de promedios":** promediar TMO de agentes sin ponderar volumen distorsiona la realidad (ej.: 15 s y 35 s dan 25 s simple, pero el valor real ponderado es 23 s).
- **Silencios y esperas ocultos:** difícil detectar a tiempo asesores con pausas prolongadas.
- **Reacción tardía:** los reportes se consolidan al final del día o la semana.

2. Objetivos y funcionalidades

Objetivo general

Automatizar el ciclo completo de ingesta, cálculo ponderado, auditoría de calidad y envío periódico de reportes de TMO, reduciendo el error humano y permitiendo decisiones inmediatas mediante alertas horarias y consultas en lenguaje natural.

Funciones principales

- **Carga de Excel** (`.xlsx` / `.xls` , máx. 50 MB) con vista previa, detección de hoja y mapeo automático de columnas por sinónimos.
- **Limpieza y validación:** detecta duplicados exactos, duraciones negativas o no numéricas y filas incompletas, sin alterar el dato original.
- **Motor de TMO:** TMO por asesor y global ponderado, % de silencio, cumplimiento y outliers por Z-score > 2.0.
- **Asistente de consultas** (motor de reglas): "¿Cómo va el agente X?", "¿Quiénes están pasados?", "¿Quién es el mejor agente?".

- **Scheduler horario** (por defecto cada 60 min, configurable) y botón de envío inmediato.
- **Reporte Excel de 4 hojas** con gráficos nativos de barras y dona.
- **Orquestación opcional con n8n** (3 workflows) e historial de ejecuciones y auditoría de errores.

Beneficios y características destacadas

- Regla determinista y reproducible: el resultado no depende de un modelo generativo.
- Funciona sin servicios externos: persistencia local en `database_local.json`; Supabase es opcional.
- Dashboard con 8 KPIs, 10 pantallas operativas y reportes descargables.

Clasificación	Condición
`APROBADO`	TMO ≤ umbral (por defecto 20.0 s)
`REQUIERE_REVISION`	TMO > umbral
`ERROR_DATOS`	Sin llamadas válidas / datos nulos o negativos
Anomalías (capa de análisis)	Outlier Z > 2.0; silencio promedio > 40 % del TMO

3. Tecnologías utilizadas

Categoría	Detalle
Lenguajes	TypeScript 5.8, JavaScript (Node.js ≥ 18), Python 3.9+ (gráficos Excel), SQL (PostgreSQL)
Backend	Express 4.21, Multer, Nodemailer 6, ExcelJS, SheetJS (`xlsx`), Zod, UUID, dotenv, CORS
Frontend	React 19, Vite 6, TailwindCSS 3.4, lucide-react
Gráficos Excel	`openpyxl` (BarChart y DoughnutChart); respaldo con ExcelJS si falta Python
Base de datos	JSON local (`database_local.json`) por defecto; PostgreSQL / Supabase (12 tablas, RLS, Storage) opcional
Externos	SMTP (Gmail), n8n (webhooks y workflows JSON). Variables para Gemini/OpenAI previstas en la configuración

> **Nota:** el módulo `aiService.ts` hoy aplica reglas analíticas locales; las claves `GEMINI_API_KEY` / `OPENAI_API_KEY` están reservadas para una integración futura con un LLM y no se invocan en el código actual.

4. Estructura del proyecto

```
PROYECTO DE IA BPO/
├── backend/                  API REST (Node + Express + TypeScript)
│   ├── src/index.ts          Arranque del servidor, CORS, /api/health, scheduler
│   ├── src/config/           Variables de entorno y cliente Supabase / BD local
│   ├── src/controllers/      Lógica por módulo (file, dashboard, call, agent, report,
│   │                         ai, settings, n8n, execution)
│   ├── src/routes/           Rutas /api y carga con Multer (límite 50 MB)
│   ├── src/middleware/       idempotency.ts y errorHandler.ts
│   ├── src/services/         tmoCalculator, excelProcessor, excelGenerator,
│   │                         aiService, emailService, schedulerService
│   ├── src/types/            Interfaces TypeScript
│   ├── scripts/              generate_excel_charts.py, test_gmail_smtp.js
│   ├── dist/                 Código compilado (npm run build)
│   ├── .env.example          Plantilla de variables de entorno
│   └── database_local.json   Persistencia local
├── frontend/                 Interfaz React + Vite + Tailwind
│   ├── src/pages/            10 pantallas (Dashboard, Carga Excel, Llamadas, Agentes,
│   │                         Reportes, Asistente IA, Umbral, Correo, Historial, Errores)
│   ├── src/components/       KPICard, BarChart, DonutChart, Navbar, Sidebar
│   ├── src/services/api.ts   Cliente HTTP tipado
│   └── vite.config.ts        Puerto 5173 y proxy /api → :4000
├── database/                 schema.sql, rls_policies.sql, functions.sql (Supabase)
├── n8n/                      README y 4 workflows JSON (3 flujos + master)
├── tests/                    run_tests.js, test_calculations.ts, test_excel_pipeline.ts
├── sample_data/              Datasets demo y casos borde + seed_sample_data.py
├── docs/                     SUPABASE_SETUP.md, VALIDATION_REPORT.md, guion de diapositivas
└── seed_live_platform.js     Carga el dataset demo a la API en ejecución
```

5. Instalación y configuración

Requisitos previos

- Node.js 18 o superior y npm
- Python 3.9 o superior con `openpyxl` 3.1+ (opcional pero recomendado)
- Navegador moderno; cuenta SMTP (Gmail con contraseña de aplicación) para envíos

Descargar el proyecto

El proyecto se distribuye como archivo comprimido (`PROYECTO_DE_IA_BPO.rar`); descomprímalo y ubíquese en la carpeta raíz. Si lo sube a un repositorio, use `git clone <URL-del-repositorio>`.

Instalar dependencias

```
# Backend
cd backend
npm install
npm run build

# Frontend
cd ../frontend
npm install

# Gráficos nativos en Excel
python -m pip install openpyxl
```

Variables de entorno

Copie `backend/.env.example` a `backend/.env` y complete los valores:

Variable	Descripción	Por defecto
`PORT`	Puerto del backend	4000
`NODE_ENV`	Entorno	development
`FRONTEND_URL`	URL del frontend	http://localhost:5173
`SMTP_HOST / SMTP_PORT / SMTP_SECURE`	Servidor de correo (Gmail: 465 + SSL)	smtp.gmail.com / 465
`SMTP_USER / SMTP_PASS`	Cuenta y **contraseña de aplicación**	(vacío)
`SMTP_FROM_NAME / SMTP_FROM_EMAIL`	Remitente	Call Center Analytics
`DEFAULT_TMO_THRESHOLD`	Umbral de TMO en segundos	20.0
`SUPABASE_URL / _ANON_KEY / _SERVICE_ROLE_KEY`	Opcionales; si faltan, se usa la BD local	—
`SUPABASE_STORAGE_BUCKET_RAW / _REPORTS`	Buckets de archivos	raw-excels / processed-reports
`AI_PROVIDER / AI_MODEL / GEMINI_API_KEY / OPENAI_API_KEY`	Reservadas para integración LLM	gemini / gemini-1.5-pro
`N8N_WEBHOOK_URL_WORKFLOW_1 / _2, N8N_API_KEY`	Webhooks de n8n	localhost:5678

> **Importante:** el `backend/.env` y el README anterior incluidos en el archivo contienen una contraseña de aplicación de Gmail en texto plano. Revóquela desde la cuenta de Google, genere una nueva y no la publique ni la suba a un repositorio.

6. Cómo ejecutar el proyecto

```
# Terminal 1 - Backend
cd backend
npm run dev            # desarrollo (ts-node-dev)
# o:  npm start        # usa dist/index.js tras npm run build

# Terminal 2 - Frontend
cd frontend
npm run dev
```

Servicio	Puerto	URL
Backend API	4000	http://localhost:4000/api
Frontend (Vite)	5173	http://localhost:5173
n8n (opcional)	5678	http://localhost:5678

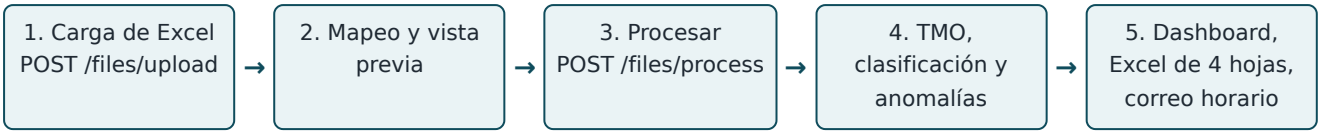
No hay URL pública ni demo desplegada.

Cómo verificar que funciona

- Abra `http://localhost:4000/api/health`: debe responder `{"status":"ONLINE", ...}`.
- En consola debe verse "SERVIDOR BACKEND INICIADO EN PUERTO: 4000".
- Con el backend activo, cargue datos demo: `node seed_live_platform.js`.
- Abra `http://localhost:5173`: el dashboard debe mostrar KPIs y gráficos; pruebe el Asistente con "Cómo va el agente Carlos Ramirez".

7. Funcionamiento del sistema

Flujo de trabajo



Entradas y salidas

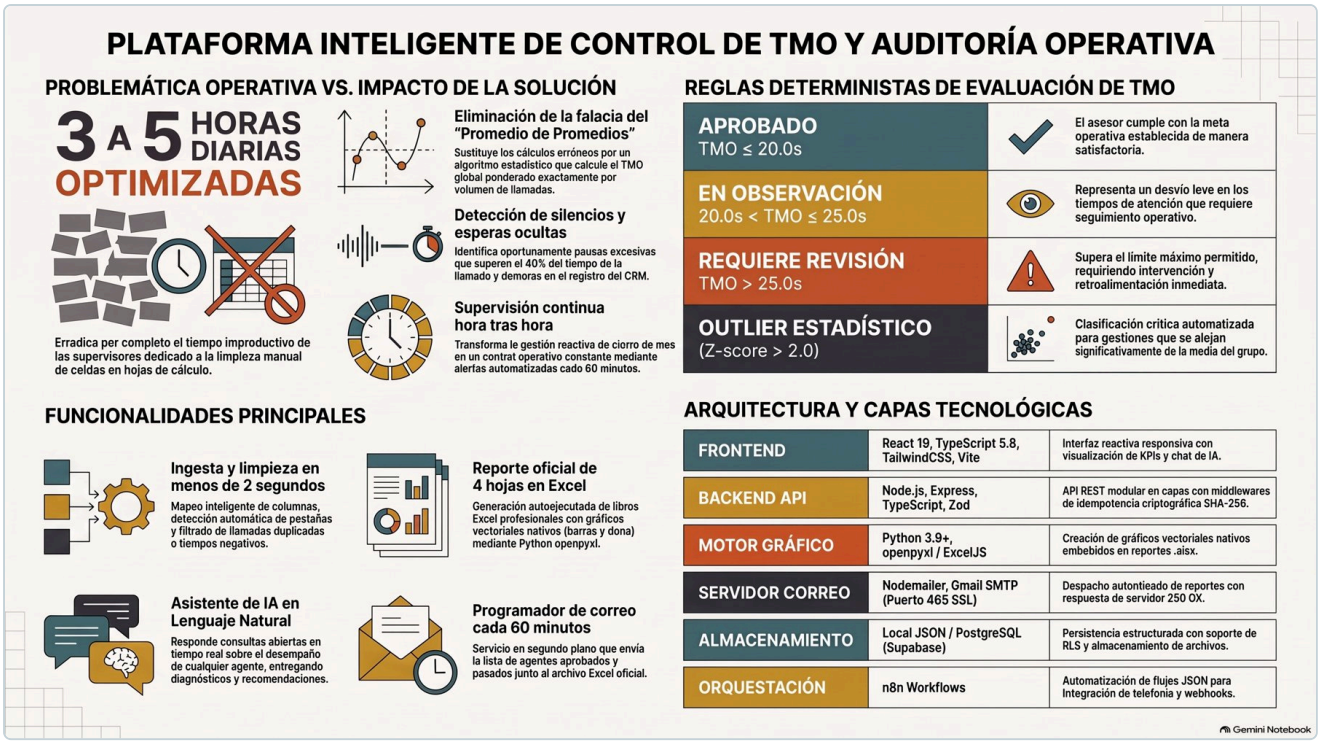
Entradas	Salidas
Excel con columnas como ID de llamada, código y nombre de agente, duración total, tiempo de silencio, estado de llamada Preguntas en lenguaje natural Umbral y correo destinatario / CC	Dashboard con 8 KPIs, gráficos de barras y dona Respuestas con diagnóstico y recomendaciones Correo HTML con tablas "pasados" y "van bien" + Excel adjunto Libro Excel: Datos Originales, Resumen de Agentes, Análisis Estadístico, Validación IA

Endpoints principales (prefijo `/api`)

Ruta	Función
`POST /files/upload · /files/process · GET /files`	Carga, procesamiento y listado de archivos
`GET /dashboard/stats · /calls · /agents · /agents/:code`	KPIs, llamadas y fichas de asesores
`GET /reports · /reports/download/:id`	Reportes Excel
`POST /ai/ask · GET /ai/validations`	Asistente y validaciones
`GET/POST /settings/threshold · /settings/email · /settings/scheduler`	Umbral, correo y programador
`POST /n8n/trigger-workflow-1 · -2 · /n8n/webhook-callback`	Integración con n8n
`GET /executions · /errors · POST /errors/:id/action`	Historial y auditoría de errores

Ejemplo de consulta: `POST /api/ai/ask` con una pregunta como "Dame el TMO del agente Carlos Ramirez" devuelve su ficha, estado, causas y plan de acción.

Vista general (infografía del proyecto)



Infografía incluida en el proyecto. Nota: la banda "En observación" mostrada allí no está implementada en el código actual.

8. Seguridad, pruebas y errores

Cómo ejecutar las pruebas

```
cd backend && npm run build     # las pruebas usan backend/dist
cd .. && node tests/run_tests.js
```

Cubren: (1) TMO individual y global ponderado (AG-01 = 15.0 s, AG-02 = 35.0 s, global = 23.0 s, no 25.0 s) y clasificación; (2) ingesta del dataset demo (≥ 80 llamadas) y generación del Excel de 4 hojas; (3) casos borde: duplicados, duraciones negativas y texto no numérico. Requiere Python con `openpyxl` para validar los gráficos nativos.

Validaciones implementadas

- Solo se aceptan archivos `.xlsx` y `.xls`, con límite de 50 MB.
- Validación de fila: código de agente obligatorio, duración numérica y no negativa, detección de duplicados.
- Idempotencia en `/files/process` mediante cabecera `X-Idempotency-Key` (o `idempotencyKey` en el cuerpo): una clave repetida no reprocesa. Si no se envía clave, la ejecución es directa.
- Manejador global de errores que registra cada fallo en la auditoría (`auditoria_errores`).
- Esquema SQL con restricciones `CHECK` y políticas RLS por rol (admin, supervisor, analista).

Errores frecuentes y soluciones

Error	Causa	Solución
`EADDRINUSE: 4000`	Puerto ocupado	Cerrar el proceso o cambiar `PORT` (y el proxy en `vite.config.ts`)
`535 5.7.8 Invalid login`	Contraseña de aplicación inválida	Generar una nueva y colocarla en `SMTP_PASS`; probar con `node backend/scripts/test_gmail_smtp.js`
`No module named 'openpyxl'`	Falta la librería	`python -m pip install openpyxl`; si no, se usa el respaldo ExcelJS
`Cannot find module ../backend/dist/...`	Backend sin compilar	`cd backend && npm run build`
"Solo se permiten archivos .xlsx o .xls"	Formato no soportado	Exportar el archivo a Excel
Pantalla sin datos	No se ha procesado ningún archivo	Cargar un Excel o ejecutar `node seed_live_platform.js`

Protección de credenciales y datos sensibles

- Las credenciales viven solo en `backend/.env`, leído por el servidor; nunca en el código React.
- Agregue a `.gitignore`: `.env`, `backend/uploads/`, `backend/reports/`, `database_local.json`.
- Use contraseña de aplicación, no la clave maestra de la cuenta.
- Los archivos cargados se conservan sin modificar para auditoría.

> **Recomendaciones de endurecimiento antes de producción:** CORS está abierto (`origin: '*'`) y la API no implementa autenticación propia; restrinja el origen, añada autenticación y use HTTPS.

9. Equipo y créditos

- **Jefferson Cacerez** — Líder de proyecto, diseño de métricas de TMO y desarrollo full stack (según el README original).
- **Equipo de Operaciones & BPO Analytics** — validación operativa de indicadores y casos de negocio.
- **Repositorio:** no definido (completar). **Demo:** no existe despliegue público.
- Material de apoyo incluido: presentación (`.pptx` / `.pdf`), infografía y guion de diapositivas en `docs/`.

10. Licencia y estado del proyecto

Licencia

El README original declara licencia **MIT**, pero el proyecto **no incluye un archivo** `LICENSE`. Para que aplique, agregue el texto de la licencia MIT con el año y el titular.

Estado

Funcional · prototipo / MVP Las pruebas automatizadas del cálculo, la ingesta y el Excel pasan según el informe de validación; falta endurecimiento de seguridad y despliegue para considerarlo listo para producción.

Mejoras futuras

- Integración real con un LLM (Gemini/OpenAI) usando las variables ya previstas.
- Implementar la banda "En observación" (20–25 s) descrita en la infografía.
- Autenticación y roles en la API; restringir CORS.
- Webhooks con marcadores telefónicos (Genesys, Asterisk/FreePBX, Vicidial).
- Alertas por WhatsApp Business y Microsoft Teams.
- Modelos predictivos de picos de TMO por franja horaria.
- Ejecutar `npm test` real (el script actual apunta a archivos `*.test.js` que no existen).